

PROJECT NAME:TIC TAC TOE

CODE:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Player class to represent a player in game
```

```
class Player {
```

```
private:
```

```
    char symbol;
```

```
    string name;
```

```
public:
```

```
    // Constructor
```

```
    Player(char sym = 'X', string n = "Player X") : symbol(sym),  
name(n) {}
```

```
    // Getter methods
```

```
    char getSymbol() const { return symbol; }
```

```
    string getName() const { return name; }
```

```
};
```

```
// Board class to manage game board
class Board {
private:
    char grid[3][3];
    int filledCells; // Counter for filled cells

public:
    // Constructor to initialize the board
    Board() : filledCells(0) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                grid[i][j] = ' ';
            }
        }
    }

    // Method to display the board
    void drawBoard() const {
        cout << "-----" << endl;
        for (int i = 0; i < 3; i++) {
            cout << "| ";
            for (int j = 0; j < 3; j++) {
```

```
        cout << grid[i][j] << " | ";  
    }  
    cout << endl << "-----" << endl;  
}  
}
```

```
// Method to check if a move is valid  
bool isValidMove(int row, int col) const {  
    return (row >= 0 && row < 3 && col >= 0 && col < 3 &&  
grid[row][col] == ' ');  
}
```

```
// Method to make a move  
void makeMove(int row, int col, char symbol) {  
    if (isValidMove(row, col)) {  
        grid[row][col] = symbol;  
        filledCells++; // Increment counter when a cell is filled  
    }  
}
```

```
// Method to check for a win  
bool checkWin(char symbol) const {
```

```
// Check rows
for (int i = 0; i < 3; i++) {
    if (grid[i][0] == symbol && grid[i][1] == symbol &&
        grid[i][2] == symbol) {
        return true;
    }
}
```

```
// Check columns
for (int i = 0; i < 3; i++) {
    if (grid[0][i] == symbol && grid[1][i] == symbol &&
        grid[2][i] == symbol) {
        return true;
    }
}
```

```
// Check diagonals
if (grid[0][0] == symbol && grid[1][1] == symbol &&
    grid[2][2] == symbol) {
    return true;
}

if (grid[0][2] == symbol && grid[1][1] == symbol &&
    grid[2][0] == symbol) {
```

```
        return true;
    }
```

```
    return false;
}
```

```
// Method to check if board is full using the counter
```

```
bool isFull() const {
    return filledCells == 9;
}
```

```
// Method to get the number of filled cells
```

```
int getFilledCellsCount() const {
    return filledCells;
}
};
```

```
// Game class to manage the game logic
```

```
class TicTacToe {
private:
    Board board;
    Player players[2];
```

```
int currentPlayerIndex;
```

```
public:
```

```
// Constructor
```

```
TicTacToe() : currentPlayerIndex(0) {  
    players[0] = Player('X', "Player X");  
    players[1] = Player('O', "Player O");  
}
```

```
// Method to get the current player
```

```
Player& getCurrentPlayer() {  
    return players[currentPlayerIndex];  
}
```

```
// Method to switch turns
```

```
void switchTurn() {  
    currentPlayerIndex = (currentPlayerIndex + 1) % 2;  
}
```

```
// Method to play the game
```

```
void play() {  
    int row, col;
```

```
cout << "Welcome to Tic-Tac-Toe!" << endl;

while (!board.isFull()) {
    // Display the board
    board.drawBoard();

    Player& currentPlayer = getCurrentPlayer();

    // Get valid input
    while (true) {
        cout << currentPlayer.getName() << " (" <<
currentPlayer.getSymbol()
        << "), enter row (1-3) and column (1-3): ";
        cin >> row >> col;
        row--; col--; // Convert to 0-indexed

        if (board.isValidMove(row, col)) {
            break;
        } else {
            cout << "Invalid move. Try again." << endl;
        }
    }
}
```

```
// Make move
board.makeMove(row, col, currentPlayer.getSymbol());

// Check for win
if (board.checkWin(currentPlayer.getSymbol())) {
    board.drawBoard();
    cout << currentPlayer.getName() << " wins!" << endl;
    return;
}

// Switch turns
switchTurn();
}

// Game ended in a draw
board.drawBoard();
cout << "It's a draw!" << endl;
}
};

int main() {
```

```
// Creating game object
TicTacToe game;

// Starting the game
game.play();
return 0;
}
```

EXPECTED OUTPUT:

Welcome to Tic-Tac-Toe!

| | | |

| | | |

| | | |

Player X (X), enter row (1-3) and column (1-3): 2 2

| | | |

| | X | |

| | | |

Player O (O), enter row (1-3) and column (1-3): 1 1

O			
---	--	--	--

	X		
--	---	--	--

--	--	--	--

Player X (X), enter row (1-3) and column (1-3): 3 1

O			
---	--	--	--

	X		
--	---	--	--

X			
---	--	--	--

Player O (O), enter row (1-3) and column (1-3): 1 3

O		O	
---	--	---	--

	X		
--	---	--	--

X			
---	--	--	--

Player X (X), enter row (1-3) and column (1-3): 3 3

| O | | O |

| | X | |

| X | | X |

Player O (O), enter row (1-3) and column (1-3): 1 2

| O | O | O |

| | X | |

| X | | X |

Player O wins!